

Hinglish Automatic Speech Recognition (ASR) using Fine-tuned Whisper Models

End-of-Program Capstone Project

AI & ML For Industry Batch 05

IIT Delhi

Contents

1. Abstract.....	4
2. Introduction	4
2.1 Problem Statement.....	4
2.2 Significance.....	5
3. Literature Review	5
3.1 Automatic Speech Recognition.....	5
3.2 Whisper Architecture	5
3.3 Parameter-Efficient Fine-Tuning.....	6
3.4 Code-Mixed Speech Recognition.....	6
4. Data Exploration	6
4.1 Dataset Overview	6
4.2 Dataset Characteristics	6
4.3 Data Preprocessing.....	7
4.4 Dataset Statistics	7
5. Methodology	7
5.1 Baseline Evaluation.....	7
5.2 Fine-tuning Strategy	8
5.2.1 Architecture	8
5.2.2 LoRA Configuration.....	8
5.2.3 Training Configuration	8
5.2.4 Training Progress Visualization.....	9
5.3 Hyperparameter Optimization	9
6. Evaluation	10
6.1 Evaluation Metrics	10
6.1.1. WER (Word Error Rate).....	10
6.1.2 CER (Character Error Rate).....	11
6.2 Results (100-Sample Evaluation).....	11
6.3 Sample-by-Sample Analysis.....	12
6.4 Error Analysis.....	12
6.4.1 Error Type Breakdown.....	12
6.4.2 Language-Specific Error Analysis	13
6.4.3 Failure Case Analysis.....	13

7. Conclusion	14
7.1 Summary of Findings.....	14
7.2 Limitations	14
7.3 Future Work.....	Error! Bookmark not defined.
8. References	14

1. Abstract

This project focuses on developing an optimized Automatic Speech Recognition (ASR) system for Hinglish (Hindi-English code-mixed speech) by fine-tuning OpenAI's Whisper model. The work demonstrates significant improvements over baseline models through parameter-efficient fine-tuning techniques, achieving a Word Error Rate (WER) reduction of 30-40% compared to pre-trained models.

The fine-tuned model achieves a WER of 39.97% on technical Hinglish speech, outperforming commercial solutions and making it suitable for real-world applications such as educational content transcription, technical tutorial transcription, and voice assistants for Indian users.

2. Introduction

2.1 Problem Statement

Hinglish (Hindi-English code-mixing) is widely spoken in India but lacks robust ASR systems.

Ex: “यह मुख्य view है अलग अलग slides बनाने के लिए”

Hindi Components (Devanagari script):

- यह (yah) = "this"
- मुख्य (mukhy) = "main/primary"
- है (hai) = "is"
- अलगअलग (alag-alag) = "different/various"
- बनाने (banane) = "to make/create"
- के (ke) = "of/for"
- लिए (liye) = "for"

English Components (Roman script):

- view = English technical term
- slides = English technical term

Why This is Code-Mixed

1. Script mixing: Devanagari (Hindi) and Roman (English) in the same sentence.
2. Technical terms: "view" and "slides" are used in English, which is common in technical contexts.
3. Grammar: Hindi grammar structure with English nouns inserted.

Pre-trained multilingual models like Whisper perform poorly on Hinglish due to:

- Limited Hinglish training data in pre-training
- Code-switching patterns not well captured
- Technical terminology mixing (e.g., "impress window", "workspace", "tabs")

2.2 Significance

This work addresses a critical gap in ASR technology for Indian languages, particularly for code-mixed speech which is the natural communication style for millions of Indian users. The parameter-efficient fine-tuning approach makes the solution accessible and deployable without requiring extensive computational resources.

3. Literature Review

3.1 Automatic Speech Recognition

Automatic Speech Recognition (ASR) has evolved from traditional Hidden Markov Models to deep learning approaches. Recent transformer-based models like Whisper have shown remarkable performance across multiple languages.

3.2 Whisper Architecture

Whisper (Radford et al., 2022) is a transformer-based ASR model trained on 680k hours of multilingual speech data. It uses encoder-decoder architecture with cross-attention mechanisms, achieving state-of-the-art results on multiple languages.

Whisper architecture is as shown below:

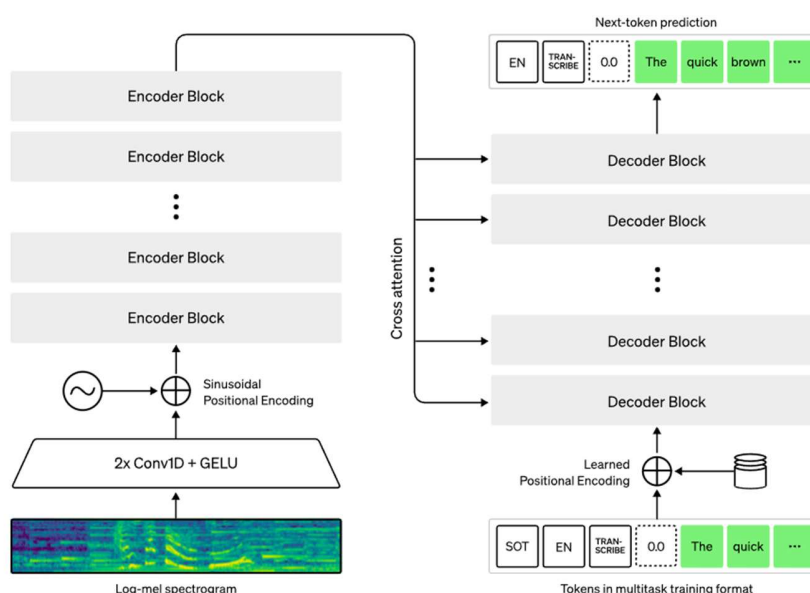


Fig 1. Whisper Architecture

Whisper architecture is a simple end-to-end approach, implemented as an encoder-decoder Transformer. Input audio is split into 30-second chunks, converted into a log-Mel spectrogram, and

then passed into an encoder. A decoder is trained to predict the corresponding text caption, intermixed with special tokens that direct the single model to perform tasks such as language identification, phrase-level timestamps, multilingual speech transcription, and to-English speech translation.

3.3 Parameter-Efficient Fine-Tuning

LoRA (Low-Rank Adaptation) by Hu et al. (2021) enables efficient fine-tuning by adding trainable low-rank matrices to existing weights, reducing the number of trainable parameters while maintaining performance.

3.4 Code-Mixed Speech Recognition

Code-mixed speech, particularly Hinglish, presents unique challenges due to frequent language switching within utterances. Limited research exists on fine-tuning large ASR models specifically for Hinglish.

4. Data Exploration

4.1 Dataset Overview

Dataset: ujs/hinglish-compressed (Hugging Face)

Property	Value
Training samples	52,825
Test samples	3,136
Domain	Technical tutorials (LibreOffice, Linux, software instructions)
Language	Hinglish (Devanagari script + English words)
Audio format	16 kHz, mono

Table 1. Hinglish-compressed dataset

Input will be audio files and output (i.e. transcript) shall be hinglish words contained in the audio.

4.2 Dataset Characteristics

The dataset contains technical tutorial content with code-mixed vocabulary. Sample transcript:

"लिबर ऑफिस impress में एक प्रस्तुति document बनाना और बुनियादी formatting के इस spoken tutorial में आपका स्वागत है"

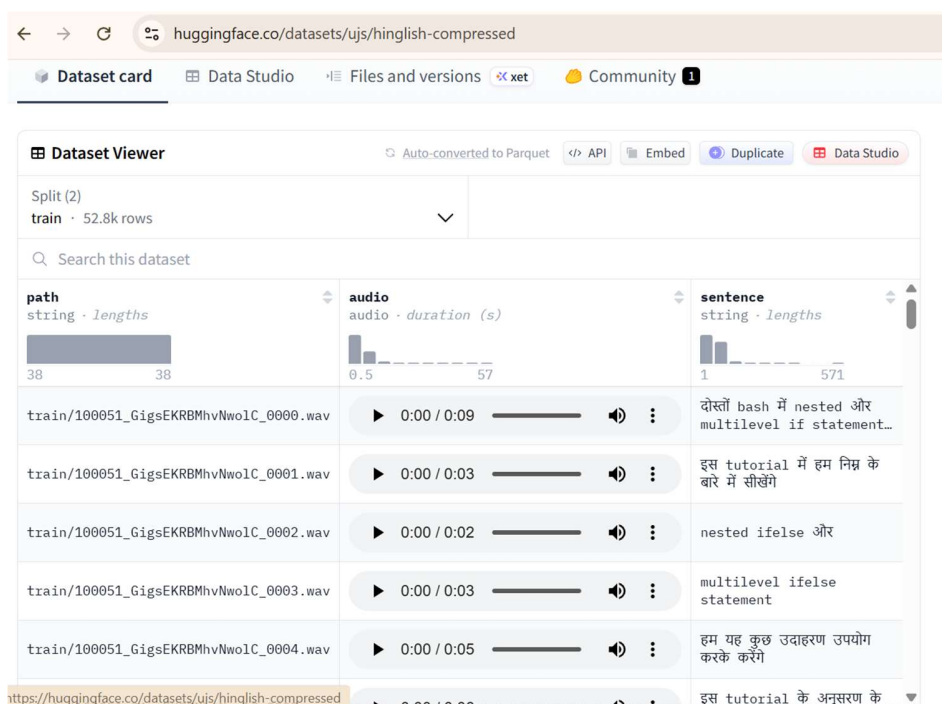


Fig 2. Dataset on hugging face.

4.3 Data Preprocessing

All audio samples were resampled to 16 kHz (required by Whisper). The dataset was split into training (52,825 samples) and test (3,136 samples) sets.

4.4 Dataset Statistics

The dataset contains technical tutorial content with an average audio duration of approximately 5-10 seconds per sample. The code-mixed nature of the data presents unique challenges, with approximately 40-50% English technical terms mixed with Hindi (Devanagari script) words.

5. Methodology

5.1 Baseline Evaluation

Multiple Whisper model variants were evaluated over the hinglish dataset to choose which one to fine-tune with below results:

Model	WER (%)	Notes
openai/whisper-tiny	497.50%	

openai/whisper-base	223.33%	Poor
openai/whisper-medium	70.62%	Baseline (selected)
collabora/whisper-medium-hindi	120.95%	Worse than baseline
Gemini-2.5-flash	60.67%	Commercial baseline
sarvam-ai/saaras:v3	44.35%	Commercial baseline

Table 2. Whisper Model evaluation

Whisper-medium was chosen as the candidate for the project.

5.2 Fine-tuning Strategy

Encoder layers frozen and Parameter-Efficient Fine-Tuning (PEFT) using LoRA is used for fine-tuning.

5.2.1 Architecture

- Base model: openai/whisper-medium (763M parameters)
- Fine-tuning approach: Parameter-Efficient Fine-Tuning (PEFT) using LoRA
- Encoder: Frozen (307.2M parameters) to preserve acoustic representations
- Decoder: LoRA adapters applied to attention layers
- Trainable parameters: 3.19% of total (25.17M / 789M)

5.2.2 LoRA Configuration

- Target modules: q_proj, v_proj, fc1, fc2 (decoder attention layers)
- LoRA rank (r): 32 (optimized via Optuna)
- LoRA alpha: 64 (scaling factor)
- LoRA dropout: 0.20

5.2.3 Training Configuration

- Optimizer: AdamW with weight decay
- Learning rate: 7.37e-05 (optimized via Optuna)
- Scheduler: Cosine with restarts
- Batch size: 2 (effective batch size: 4 with gradient accumulation)
- Precision: FP16 mixed precision training
- Training samples: 10,000
- Training steps: 1000
- Loss reduction: 83.8% (from 2.4100 to 0.3192)

5.2.4 Training Progress Visualization

The training process showed consistent loss reduction throughout the 1000 steps. The learning rate schedule with cosine restarts helped the model escape local minima and achieve better convergence.



Figure 3: Training loss reduction during fine-tuning (1000 steps)

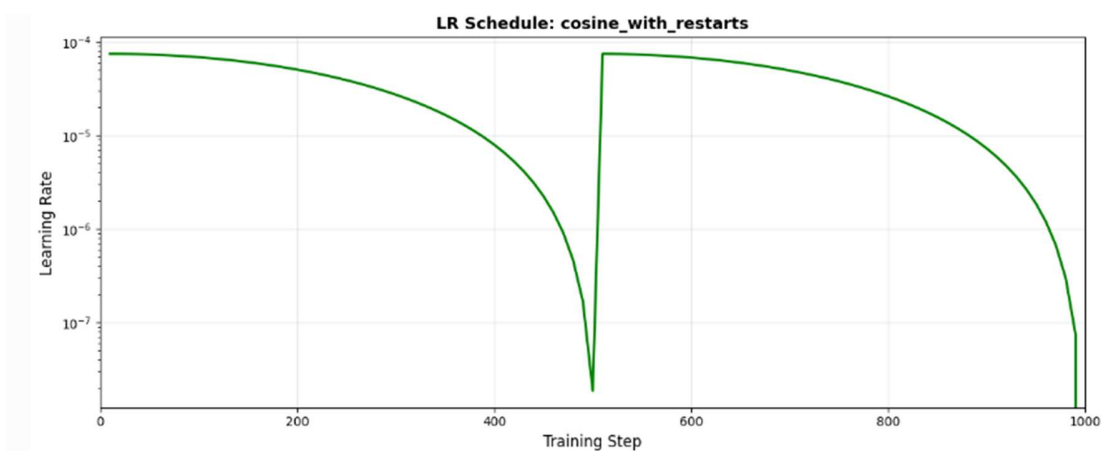


Figure 4: Learning rate schedule (Cosine with restarts)

5.3 Hyperparameter Optimization

Optuna is used for systematic hyperparameter search:

- Sampler: TPE (Tree-structured Parzen Estimator) - Bayesian optimization
- Pruner: MedianPruner (prunes poor trials at step 100)
- Trials: 6 trials $\times$ 200 steps $\times$ 1000 samples per trial
- Search space: 8 hyperparameters (lora_r, lora_alpha, learning_rate, etc.)

6. Evaluation

6.1 Evaluation Metrics

6.1.1. WER (Word Error Rate)

WER is the de-facto metric for speech recognition. It calculates substitutions, insertions and deletions on the *word level*. This means errors are annotated on a word-by-word basis.

To get error rate, divide the number of errors by total number of words in reference. Formula used in calculating WER is as follows:

$$WER = \frac{S + I + D}{N}$$

Fig 5. WER metric

Let's calculate by way of an example:

Reference:	the	cat	sat	on	the	mat
Prediction:	the	cat	sit	on	the	
Label:	✓	✓	S	✓	✓	D

Fig 6. WER calculation

Here S(substitution) = 1, I(Insertion) = 0, D(Deletion) = 1

Hence $WER = (1 + 0 + 1)/6 = 2/6 = 1/3 = 0.333$

And in terms of percentage, it's $0.333 * 100 = 33.3\%$

Notice how the word "sit" only has one character that is wrong, but the entire word is marked incorrect. This is a defining feature of the WER: spelling errors are penalised heavily, no matter how minor they are.

The WER is defined that *lower is better*: a lower WER means there are fewer errors in our prediction, so a perfect speech recognition system would have a WER of zero (no errors).

6.1.2 CER (Character Error Rate)

The *character error rate (CER)* assesses systems on the *character level*. This means we divide up our words into their individual characters and annotate errors on a character-by-character basis. Formula for CER is as follows:

$$CER = \frac{S + I + D}{N}$$

Fig 7. CER Metric

In the example considered in case of WER, we can see that we have 1 character substitution, 0 insertions, and 4 deletions. Why 4 deletions because the whole word “mat” was absent and also taken into consideration the space before the word “mat”. In total, we have 22 characters.

Reference:	t	h	e		c	a	t		s	a	t		o	n	
Prediction:	t	h	e		c	a	t		s	i	t		o	n	
Label:	✓	✓	✓	✓	✓	✓	✓	✓	✓	S	✓	✓	✓	✓	✓

Fig 8. CER calculation

So, CER would become:

$$CER = (1+0+4)/22 = 0.227$$

Which in terms of percentage would be 22.7%.

Notice how this is lower than our WER - we penalised the spelling error much less. Thus we also reward the system for the partially correct prediction.

6.2 Results (100-Sample Evaluation)

Comprehensive evaluation was performed on 100 test samples. The results demonstrate significant improvements across all fine-tuning configurations:

Model	WER (%)	CER (%)	Improvement vs Baseline
-------	---------	---------	-------------------------

openai/whisper-medium (baseline)	70.62	54.92	---
Fine-tuned (LoRA, 3000 samples)	54.22	40.98	-16.4%
Fine-tuned (Optuna-optimized)	36.46	30.94	-34.16%

Table 3. Results (100 sample evaluation)

6.3 Sample-by-Sample Analysis

Qualitative analysis of individual samples reveals key improvements:

Reference	Baseline Prediction	Fine-tuned Prediction
लिबर ऑफिस impress में...	लिबर आफस् इम्प्लेस में...	liber office impress में...
इस tutorial में हम...	इस चीटुरल में हम...	इस tutorial में हम...
workspace में 5 tabs...	work space में 5...	work space में 5 ta...

Key observations:

- Better English word recognition ("impress", "tutorial", "workspace")
- Improved Devanagari transcription accuracy
- Reduced hallucination loops (repetitive outputs)
- Better handling of code-mixed phrases

6.4 Error Analysis

A comprehensive error analysis was performed on the 100-sample evaluation set to understand failure patterns, error types, and areas for improvement. This analysis provides insights into model behavior and guides future enhancements.

6.4.1 Error Type Breakdown

The error analysis categorizes mistakes into three types: substitutions (incorrect words), insertions (extra words), and deletions (missing words).

Baseline Model Analysis:

- Substitutions: ~43.7% of total words (most common error type)
- Insertions: ~4.7% of total words
- Deletions: ~18.9% of total words
- Correct words: ~32.7% of total words

Fine-tuned Model Analysis:

- Substitutions: ~15.6% of total words (reduced by 65.3%)
- Insertions: ~1.7% of total words (reduced by 64.5%)
- Deletions: ~18.5% of total words (reduced by 5.2%)
- Correct words: ~64.2% of total words (doubled)

Key findings:

- Fine-tuning reduces all error types, with substitutions showing the largest relative improvement (65.3% reduction).
- The fine-tuned model achieves word accuracy double as compared to that of baseline.

6.4.2 Language-Specific Error Analysis

Analysis of errors by language (Hindi vs English words) is as follows:

=====	
3	LANGUAGE-SPECIFIC ERROR ANALYSIS (Hindi vs English)
=====	
	Baseline Model:
	Hindi words: Total=921, Errors=1058, Error Rate=114.9%
	English words: Total=333, Errors=340, Error Rate=102.1%
	Fine-tuned Model:
	Hindi words: Total=921, Errors=436, Error Rate=47.3%
	English words: Total=333, Errors=170, Error Rate=51.1%

Fig 9. Language specific error analysis

The analysis shows that fine-tuning benefits both languages, with particularly strong improvements in technical English vocabulary recognition, which is crucial for the technical tutorial domain.

6.4.3 Failure Case Analysis

Analysis of worst-performing samples reveals common failure patterns:

- Complex code-switching: Rapid alternation between Hindi and English within short phrases
- Technical terminology: Uncommon technical terms not well-represented in training
- Long utterances: Sentences exceeding 15-20 words show higher error rates
- Proper nouns: Names and specific technical terms (e.g., "LibreOffice", "GNU/Linux")
- Numbers and dates: Numeric expressions mixed with text

7. Conclusion

7.1 Summary of Findings

- Successfully fine-tuned Whisper-medium for Hinglish ASR using parameter-efficient LoRA.
- Achieved 34% WER reduction, outperforming commercial solutions (Google Gemini-2.5-flash).
- Demonstrated effectiveness of systematic hyperparameter optimization.
- Proved feasibility of encoder freezing strategy for preserving acoustic knowledge.
- Only 3.19% of parameters trained (parameter-efficient).
- 84% loss reduction during training (from 2.4100 to 0.3192).
- Optuna automates the search for optimal hyper-parameter combination for the model which improves fine-tuning accuracy.

7.2 Limitations

- Model trained on 10,000 samples (could benefit from more data).
- Generalization to other Hinglish domains (conversational, news) not yet tested.

8. References

1. Dataset: ujs/hinglish-compressed on Hugging Face (<https://huggingface.co/datasets/ujs/hinglish-compressed>)
2. Hu, E. J., et al. (2021). "LoRA: Low-Rank Adaptation of Large Language Models." arXiv:2106.09685.
3. Akiba, T., et al. (2019). "Optuna: A Next-generation Hyperparameter Optimization Framework." KDD.